

Introdução à programação funcional em Lean 4: Primeiros passos

Ricardo J. Franquiz Flores

2026

Sumário

1	Introdução	3
2	Conhecendo o Lean	4
3	Avaliando Expressões	5
3.1	Condicionais	6
3.2	Mensagens que Você Pode Encontrar (Erros)	7
4	Tipos	7
4.1	Type vs Prop	8
4.2	O papel dos tipos	8
5	Funções e Definições	9
5.1	Definindo Funções	10
5.2	Definindo Tipos (Criando Notações)	13
6	Estruturas	14
6.1	Exemplo Especial: Plano Cartesiano	15
6.2	Atualizando Estruturas	16

4.2 Mais sobre Estruturas	18
7 Tipos de Dados e Padrões (<i>Datatypes and Patterns</i>)	20
7.1 Matching de Padrões	22
7.2 Funções Recursivas	23
8 Polimorfismos	25
8.1 Exemplo Especial: Listas	27
8.2 Mais sobre Tipos de Dados Integrados ou Embutidos	28
8.2.1 Option	29
8.2.2 Produtos (Prod)	30
8.2.3 Somas (Sum)	35
8.2.4 * Somas (Diretas) de mais de dois elementos	36
9 Recursos Adicionais (Conveniências Adicionais)	38
9.1 Parâmetros Implícitos Automáticos	38
9.2 Definições com Matching de Padrões	39
9.3 Definições Locais	40
9.4 If Let	42
9.5 Interpolação de String	43

1 Introdução

Visto como linguagem de programação, Lean é uma linguagem funcional pura e estrita com tipos dependentes.

O que isso significa?

1. Lean é uma linguagem funcional no sentido de que o modelo de execução é inspirado pela avaliação matemática. Além disso, as funções aqui são *valores de primeira classe*. Isso significa que as funções são tratadas como se fossem igual a qualquer outro dado ou variável.

Nesse sentido. **No Lean** uma função não é apenas um bloco de instruções dadas ao computador; uma função é vista como um dado que ocupa espaço na memória.

Esta forma de tratar as funções **No Lean** faz ele processar a informação da função exatamente um argumento de cada vez. Essa forma de trabalhar é uma técnica de programação conhecida com *currying*. Nesse sentido se você fornecer apenas um argumento para uma função que exige dois argumentos, por exemplo, Lean retornará uma nova função que espera o argumento faltante.

Na prática as funções ser valores de primeira classe implica que:

- (i) Funções podem ser passadas como argumentos para outras funções.

Por exemplo:

```
def AplicDuasVezes (f : Nat -> Nat) (x : Nat )
  : Nat := f (f x)

#eval AplicDuasVezes ( . + 3) 10
-- Resultado: 16
```

Aqui `(f : Nat -> Nat)` é um tipo função, e `(x : Nat )` é um tipo natural, o segundo `Nat` representa o valor que Lean vai retornar

ao usuário.

Matematicamente `AplicDuasVezes a (• + 3) 10` se traduz em

$$f(f(x)) = f(10 + 3) = f(13) = 13 + 3 = 16$$

- (ii) Funções podem ser retornadas por outras funções. Ou seja, uma função pode produzir outra função como resultado.

Por exemplo:

```
def soma_val (n : Nat) : Nat -> Nat := fun x
  => x + n

def soma_5 := soma_val 5

#eval soma_5 3
-- Resultado: 8
```

- (iii) Funções podem ser armazenadas como estrutura de dados.

Obs.: Aqui uma estrutura de dados é qualquer coisa que agrupa valores. Por exemplo uma lista

Por exemplo:

```
def funcoes : List(Nat -> Nat) (x : Nat) :=
  [( . + 1), ( . + 2), ( .^2)]

#eval funcoes.map ( . + 2) 10
-- Resultado: [3, 4, 4]

#eval funcoes[2]!
-- Resultado 4
```

2 Conhecendo o Lean

O Lean implementa uma versão estendida do chamado *Language Server Protocol*, que fornece *feedback* enquanto o usuário digita.

3 Avaliando Expressões

Em Lean, os programas são, antes de tudo, **expressões**, e a principal forma de pensar sobre computação (cálculo) é como a avaliação de expressões para encontrar seus valores.

A **avaliação** é o processo de encontrar o valor de uma expressão (tal como se faz na aritmética).

A maioria das linguagens de programação é do tipo **imperativa**. Nesse sentido, um programa é uma série de comandos ou instruções que devem ser executadas em ordem para obter o resultado do programa. Dessa forma os programas podem ter “efeitos colaterais”, ou seja, podem acontecer coisas que não seguem o modelo de avaliação de expressões matemáticas — como por exemplo o valor referenciado por uma variável pode mudar no tempo, ou arquivos podem ser deletados.

Em Lean, programas funcionam da mesma forma que expressões matemáticas. Uma vez que uma variável recebe um valor, este não muda no tempo. Nesse sentido, avaliar uma função não pode ter efeitos colaterais. Se duas expressões têm o mesmo valor, substituir uma pela outra não fará o programa calcular resultados diferentes.

Para avaliar uma expressão, escreva `#eval` seguido da expressão que deseja avaliar. Por exemplo:

```
#eval 1 + 2
-- Resultado: 3
```

Obs.: Se colocar o cursor em cima do comando `#eval`, sem fazer clique, vai aparecer o resultado acompanhado de alguma explicação.

O Lean obedece as regras de prioridade dos operadores aritméticos. Por exemplo:

```
#eval 1 + 2 * 5    -- Resultado: 11
-- Primeiro multiplica (2 * 5 = 10), depois soma (1 + 10 = 11)
```

Diferente da maioria das linguagens de programação, Lean escreve a função

ao lado dos seus argumentos (como `f x`), enquanto geralmente se escreve `f(x)`. Por exemplo:

```
#eval String.append "Hello" "Lean"
--      funcao      argumento1  argumento2
-- Resultado: "HelloLean"
```

Obs.: `String.append` junta duas cadeias de caracteres.

Os parênteses são necessários quando o argumento de uma função deve ser calculado por meio da chamada de outra função. Por exemplo:

```
#eval String.append "great" (String.append "oak" "tree")
-- Resultado: "greatoaktree"
```

Se não usar os parênteses, Lean interpreta o segundo `String.append` como um argumento do primeiro. Nesse caso vai ser gerado um erro pois `String.append` espera por argumento caracteres e não funções.

3.1 Condicionais

Na maioria das linguagens de programação existem dois tipos de condicionais:

1. **Comandos condicionais:** “fazem alguma coisa”, ou seja, determinam qual ou quais instruções serão executadas (com base num valor booleano). Por exemplo:

```
if x > 0:
    print("positivo")
else:
    print("negativo")
```

2. **Expressões condicionais:** “produzem algum valor”, ou seja, determinam qual expressão de duas será avaliada (com base num valor booleano). Por exemplo:

```
resultado = if x > 0 then "positivo" else "negativo"
```

Aqui `if` é um valor — o valor “positivo” ou “negativo” — que é atribuído a `resultado`.

Em Lean, por ser uma linguagem funcional orientada a expressões, não existem comandos condicionais — existem apenas **expressões condicionais**. Tais expressões são escritas usando `if`, `then` e `else`:

```
#eval String.append "it is" (if 1 > 2 then "yes" else "no")
-- Resultado: "it is no"
```

3.2 Mensagens que Você Pode Encontrar (Erros)

No caso de pedir ao Lean para avaliar uma função e faltar um argumento, será produzida uma mensagem de erro. Por exemplo:

```
#eval String.append "it is"
-- Mensagem: could not synthesize a 'ToExpr', 'Repr', or
--           'ToString' instance for type String -> String
```

Essa mensagem ocorre porque funções **No Lean** que são aplicadas a apenas alguns dos seus argumentos retornam novas funções que estão aguardando o restante dos argumentos.

4 Tipos

Lean possui dois universos principais:

- **Prop** (Universo das Proposições): os elementos em **Prop** são as provas de proposições lógicas.

Exemplos de proposições lógicas: $2 + 2 = 4$, $\forall n, n + 0 = n$.

Obs.: Proposições em **Prop** são apagadas durante a compilação. Nesse sentido proposições não geram código.

- **Type** (Universo dos Tipos): os elementos dos tipos são dados computacionais, estruturas ou valores.
Exemplos de tipos: `Nat`, `List Nat`.

Todo valor **No Lean** pertence a algum tipo. Pode ser `Nat`, `Int`, `String`, `Bool`, etc.

4.1 Type vs Prop

Para um tipo em **Type**, o que é o valor importa (por exemplo 3, 4, 5 são valores distintos em `Nat`).

Para uma proposição em **Prop**, apenas o fato de existir uma prova importa. Nesse sentido duas provas da mesma proposição são idênticas pelo Lean:

```
example (p q : 1 = 1) : p = q := rfl
-- p e q sao provas da proposicao 1 = 1
-- p = q e outra proposicao, demonstrada por rfl
```

Obs.: O símbolo `:=` significa “é definido como”.

4.2 O papel dos tipos

No Lean, tipos classificam programas com base nos valores que eles podem computar. Os tipos desempenham vários papéis num programa:

1. Eles permitem que o compilador decida sobre a representação de um valor na memória.
2. Servem como forma de especificar como serão as entradas e saídas de uma função. Nesse caso o compilador irá respeitar essa especificação.
3. Ajudam na prevenção de erros potenciais. Isso reduz o número de testes necessários para um programa.
4. Eles ajudam ao Lean a automatizar a produção de códigos auxiliares que eliminam códigos repetitivos.

Todo programa em Lean deve ter um tipo. Em particular, toda expressão deve ter um tipo antes de ser avaliada. Algumas vezes será necessário fornecer o tipo explicitamente para o Lean. Isso é feito usando o operador `(:)`.

```
#eval (1 + 2 : Nat)    -- Tipo Naturais
```

No **Lean**, `Nat` é o tipo que representa os valores sem sinal, assim a subtração em `Nat` retorna zero quando o resultado seja negativo. Por exemplo:

```
#eval (1 - 2 : Nat)    -- Resposta: 0
```

Para verificar o tipo de uma expressão sem avaliá-la, usamos `#check`:

```
#check (1 - 2 : Int)    -- Resposta: 1 - 2 : Int
```

Quando não é possível atribuir um tipo a um programa, tanto `#check` como `#eval` retornam um erro.

Obs.:

- `#check` → apenas inspeciona o tipo
- `#eval` → avalia a expressão e mostra o resultado

5 Funções e Definições

No **Lean**, definições são introduzidas usando o comando `def` junto com o operador lógico `:=`. Por exemplo:

```
def hello := "Hello"
-- Esta e a definicao do nome hello. Ela sera o string "Hello".
```

Neste caso a resposta No **Lean** InfoView (janela da direita no VS Code, onde aparece o estado da “prova” que está sendo escrita) aparece $\vdash \text{String}$, que significa que nossa definição é do tipo `String`. O Lean mostra o que ele precisa para completar a definição.

No **Lean**, o símbolo `=` é usado para descrever igualdade entre expressões que já existem.

Obs.: No Lean o comando `#check` é usado para perguntar ao Lean “qual é o tipo de uma expressão?”. Este comando não avalia numericamente — retorna um valor para a expressão.

O comando `#eval` é usado para perguntar ao Lean “qual é o valor de uma expressão?”. Neste caso ele retorna um “cálculo”.

```
#check hello    -- Resposta: hello : String
#eval hello     -- Resposta: "Hello" (este e o valor da
                definicao)
```

Em algumas ocasiões, a maioria das vezes, as definições não serão tão simples. Nesses casos será necessário declarar o tipo explicitamente do que estamos querendo definir. Para isso vamos usar a seguinte estrutura de escrita:

```
def "Nome da definicao" : Tipo da definicao := "A definicao"
```

Por exemplo:

```
def lean : String := "Lean"
```

Podemos juntar agora as duas definições:

```
#eval String.append hello (String.append " " lean)
-- Resposta: "Hello Lean"
-- 0 " " e a forma de denotar um caracter vazio (espaco em
    branco)
```

Visto como função matemática, `String.append` será:

$$\text{String.append} : \text{String} \rightarrow \text{String}$$

5.1 Definindo Funções

Vamos ver algumas das diferentes formas que existem de definir funções.

(i) Usando o comando `def` com argumentos explícitos

Colocar os argumentos da função antes do tipo da definição, separados por espaços:

```
def somaum (n : Nat) : Nat := n + 1
-- nome      variavel      tipo retorno  corpo
```

InfoView: Neste caso vamos observar no tipo esperado (*Expected type*):

- $n : \text{Nat} \rightarrow \text{variável}$
- $\vdash \text{Nat} \leftarrow$ o valor que espera seja retornado

Se quiser testar a função basta digitar:

```
#eval somaum 6      -- Resposta: 7
```

Se vamos definir funções com vários argumentos ou variáveis o procedimento é similar:

```
-- (1) Argumentos do mesmo tipo
def maximum (n : Nat) (k : Nat) : Nat :=
  if n < k then k else n
-- Obs.: \textbf{No Lean} este operador exige que ambos os
-- lados
-- que serao comparados sejam do mesmo tipo

#eval maximum 3 2      -- Resposta: 3
```

```
-- (2) Argumentos de tipos diferentes
def maximum (n : Nat) (q : Rat) : Rat :=
  if n < q then q else n + 1
-- Obs.: Neste caso Lean entende que Nat contido em Rat

#eval maximum 3 (5/2)      -- Resposta: 4 = 3 + 1
```

```
-- (3) SpaceBetween
def SpaceBetween (xxx : String) (yyy : String) : String :=
  String.append xxx (String.append " " yyy)

#eval SpaceBetween "Hola" "turma"
-- Resposta: "Hola turma"
```

(ii) Passando uma função anônima

Você pode criar uma função sem nome específico. No **Lean** faremos isso usando o comando `fun`:

```
def somaum : Nat -> Nat := fun n => n + 1
--                               tipo           funcao anonima

#eval somaum 3    -- Resposta: 4
```

(iii) Passando uma função preexistente

Você pode definir uma função separadamente antes e depois usar ela para avaliar:

```
def somaum (n : Nat) : Nat := n + 1

def AvalFunc (f : Nat -> Nat) (x : Nat) : Nat := f x

#eval AvalFunc somaum 10    -- Resposta: 11
```

Obs.: Sintaxe das funções dos últimos exemplos:

```
-- Argumento de tipo funcao + funcao anonima:
def "Nome" (f : Tipo -> Tipo) := fun x => funcao de x

-- Dois argumentos:
def "Nome" (f : Tipo -> Tipo) (x : Tipo) : Tipo := f x
-- Neste caso Lean interpreta como uma funcao matematica em
  duas variaveis
```

(iv) Definindo funções como exemplos

Você pode definir funções como sendo exemplos:

```
-- (i)
def somaum : (Nat -> Nat) := fun n => n + 1
example : (Nat -> Nat) := somaum
-- "somaum" e o "corpo" da funcao
```

```
-- ": (Nat -> Nat)" declara o tipo que vem depois do :=
-- "==" significa "e definido como"

#eval somaum 4    -- Resposta: 5

-- (ii)
example : (Nat -> Nat) := fun n => n + 1
-- "example" introduz uma definicao anonima e descartavel,
-- ou seja, nao ficara disponivel depois.
-- Geralmente e usado em demonstracoes rapidas ou
-- para fazer verificacoes pontuais.
-- "def" introduz uma definicao nomeada e permanente.
```

Matematicamente, as seguintes expressões fazem a mesma coisa ($n \mapsto n + 1$):

```
fun n => n + 1
example : Nat -> Nat := fun n => n + 1
def somaum : (Nat -> Nat) := fun n => n + 1
```

Existe uma diferença somente sintática: no **def** o parâmetro aparece à esquerda do símbolo `:=`, no **example** o parâmetro aparece à direita do **fun**.

5.2 Definindo Tipos (Criando Notações)

No **Lean** os tipos também são entendidos como qualquer outra expressão. Isso significa que definições podem se referir a tipos da mesma forma como se referem a valores. Na prática isso nos permite trocar os nomes dos tipos via **def**.

Exemplo: No lugar de escrever **String** podemos usar a abreviação **Str**. Para isso basta digitar:

```
def Str : Type := String

def aStr : Str := "Isto e um exemplo"
#eval aStr    -- Resposta: "Isto e um exemplo"

#eval (fun (s : Str) => s) "Isto e um exemplo"
```

```
-- Resposta: "Isto e um exemplo"
```

Obs.: Experimentar com as “definições” para tipos pode ser complicado, alguns nomes não produzem o efeito esperado.

6 Estruturas

Quando estamos tentando resolver um problema usando programação, este problema encontra-se numa área de conhecimento específico. Tal área é conhecida como **domínio**. Por exemplo:

- Um sistema bancário → Domínio financeiro
- Um jogo de xadrez → Domínio xadrez
- Um aplicativo de entrega de comida → Domínio de logística e restaurantes

Os termos, objetos ou regras dentro de um domínio serão chamados **conceitos de domínio**. Por exemplo:

- Domínio Financeiro → Conta, cliente, saldo, etc.
- Domínio Xadrez → Tabuleiro, peça, posição, etc.

O primeiro a fazer ao escrever um programa para resolver algum problema é **identificar os conceitos do domínio do problema**. Em seguida, o seguinte a fazer é encontrar as representações adequadas para estes conceitos dentro do código. Isso vai tornar o programa:

1. Mais compreensível: o código fala a mesma língua do problema.
2. Mais organizado: cada conceito fica encapsulado na sua própria estrutura.
3. Menos propenso a erro.

Definir uma estrutura introduz um tipo completamente novo **No Lean**. Este novo tipo não pode ser reduzido a nenhum outro tipo. Múltiplas estruturas podem representar conceitos diferentes. Por exemplo, um ponto pode ser representado em duas estruturas, como pares ordenados do plano cartesiano ou como pares em coordenadas polares. Nesse caso, múltiplas estruturas que representam conceitos diferentes possuem os mesmos dados.

No Lean, os ponto-flutuante (números reais) são um tipo denominado **Float**:

```
#check 1.2    -- Resposta: 1.2 : Float
```

Quando números ponto-flutuantes são escritos com a expressão decimal, Lean identifica eles como **Float**. Caso contrário devemos identificar o tipo como queremos que Lean identifique o valor.

```
#check 0      -- Resposta: 0 : Nat
#check -1     -- Resposta: -1 : Int
```

Obs.:

```
#check (1/2)  -- Resposta: 1/2 : Nat    (CUIDADO!)
```

6.1 Exemplo Especial: Plano Cartesiano

No Lean, um ponto no plano cartesiano é uma estrutura com dois campos do tipo **Float**. Vamos chamá-los de **x** e **y**. Vamos declarar estas estruturas usando a palavra-chave **structure**:

```
structure Ponto where
  x : Float
  y : Float
```

Após essa declaração, **Ponto** se torna um novo tipo de estrutura. A forma típica de criar um valor de um tipo de estrutura é fornecer os valores para todos os seus campos dentro de chaves **{ }**:

```
-- Exemplo: Origem do plano cartesiano
```

```
def origem : Ponto := { x := 0.0, y := 0.0 }
#eval origem
-- Resposta: { x := 0.000, y := 0.000 }
-- Alternativa: { x := 0, y := 0 }
```

As estruturas existem para “empacotar” coleções de dados, identificando elas com um nome e tratando como uma única unidade. Nesse sentido a extração de dados é feita usando a chamada “notação de ponto”:

```
-- (i)
#eval origem.x    -- Resposta: 0.000 (coordenada x da origem)

-- (ii)
def P : Ponto := { x := 1.0, y := 2.5 }
#eval P.y         -- Resposta: 2.500000
```

Podemos definir operações na estrutura de “produto cartesiano” que estamos construindo:

```
-- Para somar dois Pontos:
def addPontos (P1 : Ponto) (P2 : Ponto) : Ponto :=
  { x := P1.x + P2.x, y := P1.y + P2.y }

-- Para calcular a distancia entre dois Pontos:
def distTwoPoints (P1 : Ponto) (P2 : Ponto) : Float :=
  Float.sqrt (((P2.x - P1.x)^2) + ((P2.y - P1.y)^2))
```

6.2 Atualizando Estruturas

Considere a seguinte situação:

```
def P : Ponto := { x := 1.0, y := 2.5 }
def zeroX : Ponto := { x := 0, y := P.y }
#eval zeroX    #eval P
```

Nesta situação Lean interpreta **zeroX** como um novo ponto.

O que acontece nos outros linguagem de programação?

Na programação do tipo imperativa (como C, Python, Java, etc.) a situação acima pensa na seguinte lógica:

```
Input : P := (x, y);
P.x := 0
P
Output : P = (0, y)
```

Observe que o ponto P agora é outro. No caso da programação funcional P e `zeroX` são dois pontos diferentes.

O estilo de programação do Lean apresenta algumas desvantagens. Ao usar a estratégia de copiar/colar um pedaço de código podemos gerar uma duplicação das estruturas. Isso pode criar conflitos porque o Lean vai precisar conviver com múltiplos campos do mesmo tipo.

Existem duas formas de resolver este problema:

- **Manualmente:** copiando todos os campos um a um que vão ter uma “coordenada” mudada.
- **Usando with:** este comando fará a troca automaticamente.

Exemplo. (Formulário de cadastro)

```
structure Pessoa where
  nome      : String
  idade     : Nat
  cargo     : String
  salario   : Nat
  departamento : String
  admissao  : String

def joao : Pessoa := {
  nome      := "Joao",
  idade     := 25,
  cargo     := "Tecnico Administrativo",
  salario   := 5450,
  departamento := "ICET",
```

```

    admissao      := "28.05.26"
  }

#eval joao
-- Resposta:
-- { nome := "Joao",
--   idade := 25,
--   cargo := "Tecnico Administrativo",
--   salario := 5450,
--   departamento := "ICET",
--   admissao := "28.05.26" }

-- Para Maria basta fazer:
def maria : Pessoa := { joao with nome := "Maria" }

```

Exemplo. (Plano cartesiano com with)

```

def zeroX (P : Ponto) : Ponto := { P with x := 0 }

def QT : Ponto := { x := 4.3, y := 3.4 }
#eval zeroX QT
-- Resposta: { x := 0.00, y := 3.40 }

```

4.2 Mais sobre Estruturas (*Behind the Scenes*)

No Lean Um **construtor** é uma função criada para armazenar dados numa estrutura.

Um construtor não faz nenhum tipo de processo (validação, transformação, etc.); ele simplesmente armazena dados.

Por padrão, o construtor para uma estrutura S é nomeado $S.mk$, em que mk é a abreviação de *maker*.

Exemplo. Criamos a estrutura:

```

structure Data where
  dia : Nat

```

```

mes    : Nat
ano    : Nat
dia_valido : dia >= 1 ^ dia <= 31
mes_valido : mes >= 1 ^ mes <= 12

def padZero (n : Nat) : String :=
  if n < 10 then "0" ++ toString n else toString n

instance : ToString Data where
  toString d := s! "{padZero d.dia}/{padZero d.mes}/{d.ano}"

```

(2) Aplicamos o construtor:

```

def natal := Data.mk 25 12 2025 (by simp) (by simp)
#eval natal      -- Saída: 25/12/2025
#check natal     -- Saída: natal : Data
#check Data      -- Saída: Data : Type

```

Podemos sobrescrever o nome de um construtor da seguinte forma:

Escreva dois dois-pontos (`::`) no início.

Exemplo.

```

structure Data where
  dat ::
    dia    : Nat
    mes    : Nat
    ano    : Nat
    dia_valido : dia >= 1 ^ dia <= 31
    mes_valido : mes >= 1 ^ mes <= 12

instance : ToString Data where
  toString d := s! "{d.dia}/{d.mes}/{d.ano}"

```

7 Tipos de Dados e Padrões (*Datatypes and Patterns*)

Estruturas permitem combinar múltiplos dados independentes para construir um tipo totalmente novo. Tipos como estruturas, agrupando uma coleção de valores, são chamados **Tipos Produto**.

Alguns conceitos de domínio possuem uma representação natural como estruturas. Por exemplo, o formulário de cadastro de uma pessoa feito no VSCode tem uma representação como estrutura (ver o exemplo "Formulário de cadastro" acima). Mas nem sempre é possível representar conceitos de domínio como estruturas. Por exemplo, suponha que temos um sistema no qual precisamos rastrear permissões de usuários, onde alguns usuários são proprietários de documentos, outros podem editar documentos e outros usuários apenas podem vê-los.

As estruturas são uma ótima forma de manter um conjunto fixo de campos. Mas estruturas não possuem uma maneira fácil de codificar múltiplas escolhas. Ou seja, quando um dado pode ser uma coisa ou outra, estruturas passam a ser complexas para o tratamento dos dados.

Exemplo. Uma calculadora possui vários operadores binários, como soma, subtração e multiplicação. Dessa forma uma calculadora possui múltiplas escolhas para uma expressão:

$$\text{Dados} = \text{Expressões} \left\{ \begin{array}{lll} 2, 3 & \rightarrow & \text{número} \\ 2 + 3 & \rightarrow & \text{soma de números} \\ 2 - 3 & \rightarrow & \text{diferença de números} \\ (2+3) * 4 & \rightarrow & \text{multiplicação de números} \\ (2*5) + (4*5) & \rightarrow & \text{outros} \end{array} \right.$$

Além disso, note que uma expressão pode conter outras expressões dentro dela. Por exemplo, $(2*3) + (4*5)$ ou $(2+3) * 4$.

Isso torna a expressão **recursiva**, pois uma expressão é definida dentro de outra.

Tipos de dados que permitem escolhas são chamados de **Tipo Soma**. Tipos de dados que podem incluir instâncias dentro deles mesmos (expressões contendo outras expressões) são chamados de **Tipo Recursivo**. Também podemos ter tipos de dados que sejam uma combinação dos tipos mencionados acima. Tipos de dados soma recursivos são chamados **Tipo Indutivo**. Isso é devido a que a indução matemática pode ser usada para provar afirmações sobre eles.

Exemplo. Agrupando os dados que possui uma calculadora:

```
inductive Expr where
  | num : Nat -> Expr          -- numeros
  | add : Expr -> Expr -> Expr -- soma
  | mul : Expr -> Expr -> Expr -- multiplicacao
  | sub : Expr -> Expr -> Expr -- diferenca
```

Temos criado um novo tipo para classificar as instâncias que pode ter uma calculadora. Este é um tipo indutivo.

Da mesma forma como numa declaração de estrutura, coloca seus nomes num *namespace* (espaço de nomes) nomeado pelo tipo declarado.

Pessoa (estrutura) $\Rightarrow$ campos ficam identificados :

Pessoa.nome, Pessoa.idade

Nos tipos indutivos os nomes quase sempre são identificados da mesma forma, isto é: "Nome do Tipo"."Nome do construtor".

Exemplo.

```
def aval : Expr -> Nat
  | Expr.num n      => n
  | Expr.add e1 e2 => (aval e1) + (aval e2)
  | Expr.mul e1 e2 => (aval e1) * (aval e2)
  | Expr.sub e1 e2 => (aval e1) - (aval e2)
```

7.1 Matching de Padrões

No **Lean** o *matching* de padrões é um recurso, uma estratégia de programação que permite que, dentro de um programa, analise o tipo de dado e, dependendo do tipo, execute outra instrução que for precisa e também extraia os dados armazenados que estão dentro daquele tipo.

O `match` serve para descobrir e lidar com qual tipo/construtor um dado foi criado.

Vejamos um exemplo de uma função de matching:

Exemplo.

```
def isZero (n : Nat) : Bool :=
  match n with
  | Nat.zero    => true
  | Nat.succ k => false

#eval isZero 3  -- Resposta: false
```

Outra forma:

```
def isZero (n : Nat) : String :=
  match n with
  | Nat.zero    => "e zero"
  | Nat.succ k => "e sucessor de " ++ toString k

#eval isZero 3  -- Resposta: "e sucessor de 2"
```

A expressão `match` recebe o argumento `n` da função para ser desestruturado. Se `n` foi construído por `Nat.zero`, então o primeiro match é selecionado (`true` ou `"e zero"`). Se `n` foi construído por `Nat.succ`, então o segundo match é selecionado.

Observação: quando se fala em um dado o valor construído, refere-se a valores criados. Por exemplo:

```
meuNum := Nat.succ (Nat.succ Nat.zero)
```

Aqui o valor 2 foi construído usando sucessor duas vezes.

Exemplo. Criando um `match` para definir predecessores em `Nat`:

```
def pred (n : Nat) : Nat :=
  match n with
  | Nat.zero    => Nat.zero
  | Nat.succ k => k

#eval pred 8 -- Resposta: 7
```

Observação:

1. Diferença entre `#check` e `match`

O `match` é uma expressão que faz parte de algum código que estejamos escrevendo; com ele podemos tomar decisões baseadas num valor dado. Este é uma parte lógica do programa.

O `#check` é um comando para identificar o tipo de alguma estrutura que estamos trabalhando; este não faz parte do código que estamos escrevendo.

2. Diferença entre `match` e `if/else`

O `if/else` também serve para tomar decisões, porém o `match` é mais poderoso pois enquanto `if/else` testa expressões booleanas, `match` testa padrões (construtores). Também `if/else` se limita a 2 casos, já o `match` pode testar quantos casos você quiser. `If/else` não trabalha com tipos soma, já o `match` trabalha com estes tipos e permite extrair os dados construídos.

7.2 Funções Recursivas

Definições recursivas são definições que vão se referir a si mesmas, ou seja são um tipo de definições que chamam a si mesma.

Exemplo.

- Tipos de dados indutivos podem ser recursivos. `Nat` é um tipo indutivo que é recursivo, pois para definir `succ` precisa de outro `Nat`.
- Tipos de dados recursivos podem ser arbitrariamente grandes, unicamente limitados por fatores técnicos como a memória.
- Tipos de dados recursivos são complementados por funções recursivas, isto é, funções que são definidas chamando-se a si mesmas.

Exemplo. Definindo uma função recursiva que decide se um número é par:

```
def isEven (n : Nat) : Bool :=
  match n with
  | Nat.zero    => true
  | Nat.succ k => not (isEven k)
```

[Passo a passo]

$$\begin{aligned}
 \text{isEven } 3 &\rightarrow \text{not}(\text{isEven } 2) \\
 &\rightarrow \text{not}(\text{not}(\text{isEven } 1)) \\
 &\rightarrow \text{not}(\text{not}(\text{not}(\text{isEven } 0))) \\
 &\rightarrow \text{not}(\text{not}(\text{not}(\text{true}))) \\
 &= \text{false}
 \end{aligned}$$

Ramos não recursivos no código são chamados **caso base**.

Def.: Recursão Estrutural é uma técnica de programação onde uma função se chama a si mesma apenas em partes “menores” (ou “mais internas”) da estrutura de dado de entrada. Algumas vezes esta técnica é usada em demonstrações matemáticas.

O exemplo anterior usa a técnica de recursão estrutural.

No **Lean** toda função recursiva alcançará o caso base. Isso elimina a possibilidade de aparecerem loops infinitos.

Exemplo. Código de loop infinito onde o Lean mostra uma mensagem de erro:


```
def evenLoops (n : Nat) : Bool :=
  match n with
  | Nat.zero    => true
  | Nat.succ k => not (evenLoops n)  -- erro: recursao sobre n,
                                     nao k
```

Este programa não vai funcionar no Lean pois a recursão está sendo feita sobre o mesmo n e não sobre uma estrutura menor.

Exemplo. Definindo a soma de dois números usando recursão estrutural:

Note que a despeito a adição receber dois argumentos (n, r) e devolver um $(n + r)$, somente um desses argumentos precisa de inspeção:

```
def som (n : Nat) (r : Nat) : Nat :=
  match r with
  | Nat.zero    => n
  | Nat.succ k => Nat.succ (som n k)
```

Assim, por exemplo:

$$\begin{aligned}
 \text{som } 3 \ 2 &\xrightarrow{n=3, r=2=\text{succ } 1} \text{Nat.succ}(\text{som } 3 \ 1) \\
 &\xrightarrow{n=3, r=1=\text{succ } 0} \text{Nat.succ}(\text{Nat.succ}(\text{som } 3 \ 0)) \\
 &\xrightarrow{n=3, r=0} \text{Nat.succ}(\text{Nat.succ}(3)) \\
 &= \text{Nat.succ}(4) = 5
 \end{aligned}$$

8 Polimorfismos

No Lean O Polimorfismo é uma estratégia de programação onde tipos de dados e definições recebem tipos como argumentos. No caso das definições, estas podem trabalhar com vários tipos. Assim, no lugar de escrever uma função para cada tipo, vamos escrever uma função para todos os qualquer tipo.

Exemplo. Sem polimorfismo podemos definir coordenadas unicamente para

cada tipo de argumento:

```
-- Nos naturais
structure PontoNat where
  x : Nat
  y : Nat

-- Nos reais
structure PontoRe where
  x : Float
  y : Float

-- Nos inteiros
structure PontoInt where
  x : Int
  y : Int
```

Se queremos escrever esta estrutura de ponto para qualquer tipo de argumento usamos uma estrutura polimórfica:

```
structure PPonto (a : Type) where
  x : a
  y : a
  deriving Repr -- converte os pontos definidos para String

def P1 : PPonto Nat    := { x := 1, y := 2 }
def P2 : PPonto Float := { x := 3.14, y := 0.5 }
def P3 : PPonto Int    := { x := -1, y := 1 }
def P4 : PPonto String := { x := "coord1", y := "coord2" }
```

Tipos são expressões comuns em Lean, então passar argumentos a tipos polimórficos não vai precisar de alguma sintaxe especial.

Exemplo.

```
def replaceX (a : Type) (point : PPonto a) (newX : a) : PPonto
  a :=
  { point with x := newX }
```

A função `replaceX` substitui o campo `x` de um `PPonto` por um novo valor (`newX`) do mesmo tipo.

```
#eval replaceX String P4 "troca"
-- Saída: { x := "troca", y := "coord2" }

#eval P4 -- { x := "coord1", y := "coord2" }
```

Observe que quando os tipos dos argumentos `point` e `newX` mencionam α , estes argumentos estão se referindo a qualquer tipo que for fornecido no primeiro argumento.

Resumindo:

No Lean, Polimorfismo é uma estratégia de programação, mas de forma geral tipos podem ser manipulados como qualquer outro valor. Podemos passá-los como argumentos, armazená-los como variáveis ou computá-los usando `match`.

Exemplo.

```
inductive Sign where
  | pos
  | neg

def PosOrNegNumb (s : Sign) (n : Nat) :=
  match s with
  | Sign.pos => (n : Nat)
  | Sign.neg => (-n : Int)

#eval PosOrNegNumb Sign.neg 124
-- Saída: -124
```

8.1 Exemplo Especial: Listas

A biblioteca padrão do Lean inclui um tipo canônico para listas, chamado `List`.

Exemplo.

```
def PrimosMenor10 : List Nat := [2, 3, 5, 7]
```

Nos bastidores, `List` é um tipo indutivo, definido usando polimorfismo:

```
inductive List (a : Type) where
| nil   : List a                -- lista vazia
| cons  : a -> List a -> List a -- construtor para preencher
                                   a lista
```

Uma lista com n entradas contém n construtores `cons`; o último dos quais será `nil`.

Exemplo.

```
def PrimUnder10 : Lista Nat :=
  Lista.cons 2 (Lista.cons 3 (Lista.cons 5 (Lista.cons 7
    Lista.nil)))

#eval PrimUnder10
-- Saída: Lista.cons 2 (Lista.cons 3 (Lista.cons 5 (Lista.cons
  7 (Lista.nil)))
-- Esta lista é equivalente a [2, 3, 5, 7]
```

A definição da função que calcula o comprimento de uma lista é uma função polimórfica (porque recebe o tipo das entradas da lista como argumento) e recursiva (porque vai se referir a ela mesma):

```
def length (a : Type) (xs : List a) : Nat :=
  match xs with
| List.nil      => Nat.zero
| List.cons y ys => Nat.succ (length a ys)
```

Vamos escrever esta função de outra forma:

```
def length (a : Type) (xs : List a) : Nat :=
  match xs with
| []      => 0
| y :: ys => Nat.succ (length a ys)
```

8.2 Mais sobre Tipos de Dados Integrados ou Embutidos

Vejamos outras estruturas e tipos de dados indutivos que existem no Lean.

8.2.1 Option

Muitas operações em coleções de dados podem falhar ao não encontrar o que estão procurando. Muitas vezes isso se deve a que não existe o valor procurado. Isso obriga à existência de uma maneira de sinalizar a não existência de uma entrada na coleção com tal valor.

No **Lean** existe um tipo de dado chamado `Option` que equipa algum outro tipo com um indicador para valores ausentes:

```
inductive Option (a : Type) : Type where
  | none : Option a
  | some : (val : a) -> Option a
```

`Option` possui dois construtores, `some` e `none`, que representam as versões de não-nula e nula, respectivamente, para o tipo subjacente.

Exemplo. Programa para encontrar o primeiro elemento de uma lista:

```
def List.cab? (a : Type) (xs : List a) : Option a :=
  match xs with
  | []      => none
  | y :: _ => some y

def explicitPrimosUnder10 := [2, 3, 5, 7]

#eval List.cab? Nat explicitPrimosUnder10 -- some 2
#eval [2,3,5,7].cab? (a := Int)           -- some 2
#eval explicitPrimosUnder10.cab? (a := Nat) -- some 2
#eval [].cab? (a := Int)                  -- none
```

Observação: Um sublinhado é usado para representar a cauda da lista. Usar sublinhados em vez de nomes é uma maneira de comunicar que parte da entrada é ignorada. O sublinhado (`_`) num padrão `match` está dizendo: “Eu sei que tem alguma coisa aqui, mas não interessa, logo nem vou mencionar ou dar nome.”

```
def List.head? (a : Type) (xs : List a) : Option a :=
  match xs with
  | []      => none
```

```
| y :: _ => some y -- sublinhado ignora a cauda, somente
                     interessa a cabeça
```

8.2.2 Produtos (Prod)

Prod é a abreviação de ‘produto’ (*Product*). Esta é uma forma de juntar dois (ou mais valores) não necessariamente do mesmo tipo.

Esta é uma estrutura padrão do Lean definida da seguinte forma:

```
structure Prod (a : Type) (b : Type) : Type where
  fst : a
  snd : b
```

Exemplo. No começo da seção de Polimorfismos definimos a estrutura genérica de “ponto” como um par ordenado de qualquer tipo possível:

```
structure PPonto (a : Type) where
  x : a
  y : a
```

Com essa estrutura, um ponto poder ser escrito de duas formas:

```
def p1 : PPonto Int := PPonto { x := 3, y := -5 }
def p1 : PPonto Int := PPonto.mk 3 (-5)

#eval p1 -- { x := 3, y := -5 }
#eval PPonto.mk 3 (-5) -- { x := 3, y := -5 }
```

Esta estrutura de PPonto pode ser redefinida usando Prod:

```
#eval Prod.mk 3 (-5) -- Saída: (3, -5)
```

```
def p2 : Prod Nat Nat := { fst := 3, snd := 2 }
#eval p2 -- Saída: (3, -2)
#check Prod.mk 3 (-5) -- Saída: (3, -5) : Nat x Nat
```

Observação: É possível usar `#eval` com `PPonto` sem especificar o tipo ao qual pertencem as coordenadas. Tal e como foi descrito no começo do exemplo, a

expressão `#eval PPonto.mk 3 5` no Lean retorna um erro pois Lean não sabe como imprimir na tela este ponto.

Usando `deriving Repr` podemos evitar ter que especificar o tipo das coordenadas. Basta definir a estrutura:

```
structure PPonto (a : Type) where
  x : a
  y : a
  deriving Repr
```

Adicionando esta linha, o Lean ignora os tipos das coordenadas em `PPonto`.

Uma vantagem de `Prod` para `PPonto` (como foi definida) é que com `Prod` podemos formar pares ordenados em que cada coordenada é de um tipo diferente sem usar `deriving Repr`.

Exemplo.

```
def fives : Prod String Int := { fst := "five", snd := 5 }
#eval fives -- ("five", 5)
#eval Prod.mk "five" 5 -- ("five", 5)
```

O tipo `Prod  $\alpha$   $\beta$` é tipicamente escrito como $\alpha \times \beta$. Nesse caso o exemplo anterior também pode ser escrito como:

```
def fives : String x Int := { fst := "five", snd := 5 }
```

Produtos Diretos de mais de dois elementos:

- **Estrutura indutiva para produto direto de n elementos:**

Podemos construir uma família de tipos representada por tuplas de tamanho arbitrário em que cada entrada é um tipo (não um valor). Podemos fazer isso criando uma estrutura recursiva.

Ao fazer isso vamos encontrar um detalhe importante. **No Lean** tipos tem vários níveis. Tipos que falam sobre tipos “sobem” de nível.

`Type` é o tipo de nível 0 (`Nat`, `String`, `Bool`, `ListNat`,...)

`Type 1` é o tipo de nível 1 (`Type`, `ListType`, `Nat → Type`,...)

`Type 2` é o tipo de nível 2 (`Type 1`, `List(Type 1)`,...)

Cada nível o Lean vai chamar de *universo*.

`Type` é o universo dos tipos “pequenos”. Por exemplo `Nat`, `Bool`, `String`. O próprio `Type` não está em `Type` porque isso produz um paradoxo.

`List`, visto como função, é da forma `List : Type → Type`. Logo `List` é um tipo que opera no universo `Type 1`. Observe que `List Type` não pode ser uma estrutura que opera no universo `Type`, pois `Type` mora no universo `Type 1`. `List Type` é uma aplicação de `List` a `Type`. `List` sempre espera um valor em `Type` mas ao aplicar `Type` a `Type` não vamos obter um objeto em `Type`.

Assim o tipo indutivo para N elementos será:

```
inductive ProdN : List Type -> Type 1 where
| nil    : ProdN []
| cons   : {tipos : List Type} -> (a : Type) -> a -> ProdN
        tipos ->
        ProdN (a :: tipos)
```

Se a lista de tipos é vazia (`[]`) então este produto tem 0 elementos. Se a lista de tipos é `a :: tipos`, então o primeiro elemento tem tipo `a` e o resto (`ProdN tipos`) tem os tipos da lista `tipos`.

Exemplo. Vamos construir `ProdN [Nat, String, Bool, Int]`:

```
-- Passo 0: Lista vazia
def passo0 : ProdN [] := ProdN.nil

-- Passo 1: Adicionar Bool na lista vazia do passo 0
def passo1 : ProdN [Bool] := ProdN.cons Bool true passo0
```



```

-- Passo 2: Adicionar String na lista [Bool]
def passo2 : ProdN [String, Bool] := ProdN.cons String "oi"
    " passo1

-- Passo 3: Adicionar Nat na lista [String, Bool]
def passo3 : ProdN [Nat, String, Bool] := ProdN.cons Nat 3
    passo2

-- Passo 4: Adicionar Int na lista [Nat, String, Bool]
def passo4 : ProdN [Nat, String, Bool, Int] := ProdN.cons
    Int (-2) passo3

```

- **Definindo Tuplas usando Prod**

Vamos criar um programa que forma n -tuplas com todos seus elementos do mesmo tipo usando `Prod`. Para isso vamos precisar de duas funções:

1. Função `TuplaType` que vai definir a tupla homogênea (todas as entradas do mesmo tipo).
2. Função `ToTupla` cuja entrada é um tipo e um número natural, e a saída serão os valores da tupla.

Além de `Prod` vamos precisar usar constantes que não foram trabalhadas anteriormente. `Fin N` é o tipo dos números naturais menores que N . Assim:

- `Fin 0` $\rightarrow$ Nenhum número (vazio)
- `Fin 1` $\rightarrow \{0\}$
- `Fin 2` $\rightarrow \{0, 1\}$
- `Fin 3` $\rightarrow \{0, 1, 2\}$
- $\vdots$

`PUnit` é um tipo que contém exatamente um único valor, chamado `PUnit.unit` (ou `()`). Este tipo é usado quando precisamos de um tipo “vazio”, ou seja, sem informação. Este recurso existe em outras linguagens de programação; por exemplo, em Python seria `NoneType` e o valor seria `none`.

Função `TuplaType`:

```
def TuplaType (a : Type) : Nat -> Type
| 0      => PUnit
| 1      => a
| n + 1 => Prod a (TuplaType a n)
```

Observação: O Lean não vai nos retornar alguma coisa se usar `#eval` com essa função. Isso acontece porque `#eval` funciona para valores computáveis e tipos não são valores neste sentido.

Função `ToTupla`:

```
def ToTupla {a : Type} : (n : Nat) -> (Fin n -> a) ->
  (Fin n -> TuplaType a n)
| 0,      f => PUnit.unit
| 1,      f => f 0
| n + 2, f =>
  let head := f 0
  let tail := ToTupla (n + 1) (fun i => f (i + 1))
  (head, tail)
```

Observação: A expressão `a : Type` permite definir o tipo de forma implícita. Dessa forma o Lean vai deduzir sozinho o tipo com o qual está trabalhando.

`let` é uma estrutura sintática usada para declarar variáveis locais no Lean. Diferente de `def`, que trabalha com um valor global.

Exemplo. Criando a tupla (usando `Prod`):

```
def minhaFunc : Fin 3 -> Nat
```

```

| 0 => 10
| 1 => 20
| 2 => 30

def minhaTupla : TuplaType Nat 3 := ToTupla 3 minhaFunc

#eval minhaTupla -- Saída: (10, 20, 30)

```

8.2.3 Somas (Sum)

No **Lean** `Sum` é um tipo de dado que permite a escolha entre valores de dois tipos diferentes. Este tipo é definido da seguinte forma:

```

inductive Sum (a : Type) (b : Type) : Type where
| inl : a -> Sum a b
| inr : b -> Sum a b

```

`inl` é a abreviação em inglês para injeção esquerda e `inr` é a abreviação em inglês para injeção direita. Vamos denotar, usualmente, `Sum  $\alpha$   $\beta$` por $\alpha \oplus \beta$. Não existe uma sintaxe especial para os construtores `Sum.inl` e `Sum.inr`.

Exemplo.

```

-- (1)
def valor      : Sum Nat String := Sum.inl 42
def valortex   : Sum Nat String := Sum.inr "quarenta e dois"
#eval (valor, valortex) -- Saída: (42, "quarenta e dois")
#check valor      -- Saída: valor : Nat (+) String

-- (2) Usando notacao (+)
def valor      : Nat (+) String := Sum.inl 100
def valortex   : Nat (+) String := Sum.inr "cem"

-- (3) Funcao para definir uma divisao "inteira"
def divide (a b : Nat) : Nat (+) String :=
  if b == 0 then
    Sum.inr "Erro! divide por zero"
  else

```

```

Sum.inl (a / b)

#eval divide 10 5    -- Saída: 2
#eval divide 10 0    -- Saída: "Erro:..."
#eval divide 5 10    -- Saída: 0

```

8.2.4 * Somas (Diretas) de mais de dois elementos

Vamos construir um programa que define a soma de mais de dois elementos. `Sum` e `Prod` são conceitos duais (no mesmo sentido da álgebra e teoria de categorias). Dessa forma este programa se constrói com uma ideia similar a como foi construído o produto.

Vamos precisar criar duas funções:

1. Função `SumType` que define o tipo Soma aninhado.
2. Função `toSum` que qualquer índice (de 0 a $n - 1$) tem um valor de algum tipo escolhido.

```

-- (1) Funcao SumType
def SumType (a : Type) : Nat -> Type
| 0      => PUnit
| n + 1 => Sum a (SumType a n)

-- (2) Funcao toSum
def toSum {a : Type} : (n : Nat) -> (idx : Fin n) -> a ->
  (SumType a n)
| 0,      idx, _    => idx.elim0
| n + 1, idx, val =>
  if h : (idx.val) = 0 then
    Sum.inl val
  else
    let nextIdx : Fin n := ⟨idx.val - 1, by
      have h_pos : idx.val > 0 := Nat.pos_of_ne_zero h
      exact Nat.sub_lt_left_of_lt_add h_pos idx.isLt⟩
    Sum.inr (toSum n nextIdx val)

```

A parte do código com definição local `nextIdx` fará o seguinte

```
let nextIdx : Fin n := ⟨idx.val - 1, by
  have h_pos : idx.val > 0 := Nat.pos_of_ne_zero h
  exact Nat.sub_lt_left_of_lt_add h_pos idx.isLt⟩
Sum.inr (toSum n nextIdx val)
```

Se o índice for > 0 , esta função vai reduzir o índice e colocar à direita (usando `sum.inr`)

`Nat.pos_of_ne_zero` se refere à propriedade "Positive of Not Equal to Zero". Ou seja, é a prova de que se um número não é zero então é maior que zero.

`Nat.sub_lt_left_of_lt_add h1 h2`, em que $h1 : n < k$ e $h2 : k < n + m$, se refere à prova do lema "se $n \leq k$ e $k \leq n + m$ então $k - n < m$."

`idx.isLt` é a prova de $idx.val < n + 1$. Aqui `isLt` se refere a "is Less than" ("é menor que").

Vamos entender o código `toSum`: Lembre que estamos querendo definir $A_0 \oplus A_1 \oplus A_2 \oplus \dots \oplus A_n$.

`toSum` é uma função definida implicitamente com algum tipo a . A entrada desta função será uma tripla (n, idx, a) , em que `idx` denota o índice, e retorna `SumType(a, n)`.

Se $n = 0$, para qualquer `idx` e qualquer a , `toSum(0, idx, a)` retornará erro!

Para o caso em que $n > 0$, `idx`, a , tem-se que:

Se $h = 0$, ou seja, se o índice $idx.val = 0$, então coloque o valor `val` nessa posição (A_0) usando `Sum.inl val`.

Se não, se $h \neq 0$, estamos no lado restante da soma $(A_1 \oplus A_2 \oplus \dots \oplus A_n)$. Seja `nextIdx` um valor em `Fin N` definido pela posição menos um, ou seja, $nextIdx = idx.val - 1$. Afirmamos que `nextIdx` pertence ao conjunto `Fin N`, ou seja, $nextIdx < n$.

De fato observe que, pelo lema `Nat.pos_of_ne_zero h`, temos que $idx.val > 0$. Logo, aplicando o lema `Nat.sub_lt_left_of_le_add` para as hipóteses `h_pos` e `idx.isLt` (em que `idx.isLt` é a prova da afirmação $idx.val < n + 1$). Segue que $nextIdx < n$. Logo a afirmação está provada.

Assim, se o índice `idx.val > 0` então coloque o valor *val* do lado direito da soma. Como isso vale para todos os índices maiores que zero, vamos preencher o resto dos lugares ($A_1 \oplus A_2 \oplus \dots \oplus A_n$) com os respectivos valores, usando de forma recursiva o código

```
Sum.inr(toSum n nextIdx val).
```

9 Recursos Adicionais (Conveniências Adicionais)

No **Lean** existem uma série de recursos, funções ou ferramentas que reduzem esforço mental e dão mais conforto ao usuário.

9.1 Parâmetros Implícitos Automáticos

Ao escrever funções polimórficas no Lean (isto é, funções definidas com vários tipos não necessariamente iguais) não sempre é necessário listar todos os parâmetros de forma explícita. Algumas vezes o Lean reconhece automaticamente o parâmetro implícito.

Exemplo. A função comprimento de uma lista pode ser escrita de duas formas:

```
-- Forma 1: parametro explicito
def length {a : Type} (xs : List a) : Nat :=
  match xs with
  | []      => 0
  | y :: ys => Nat.succ (length ys)

-- Forma 2: parametro implicito automatico
def length (xs : List a) : Nat :=
  match xs with
  | []      => 0
  | y :: ys => Nat.succ (length ys)
```

9.2 Definições com Matching de Padrões

No Lean Ao definir funções usando `def` e envolvendo `match` podemos trocar a sintaxe de forma que não seja necessário nomear o argumento da função.

Para fazer isso o primeiro a fazer é mover os tipos dos argumentos para a direita, de modo que o retorno seja um tipo função. Em seguida, substitua `:=` por cada caso do matching e NÃO escreva `match_with`.

Exemplo. Função `length` sem uso do `match` explícito:

```
-- Com match
def length (a : Type) (xs : List a) : Nat :=
  match xs with
  | []      => 0
  | y :: ys => Nat.succ (length ys)

-- Sem match (estilo equacional)
def length : List a -> Nat
  | []      => 0
  | y :: ys => Nat.succ (length ys)
```

Exemplo. Criando uma função soma de naturais:

```
-- Forma 1
def soma (n : Nat) (m : Nat) : Nat :=
  match n with
  | 0      => m
  | k + 1 => Nat.succ (soma k m)

-- Forma 2 (estilo equacional)
def soma : Nat -> Nat -> Nat
  | 0,      m => m
  | k + 1, m => Nat.succ (soma k m)
```

Nomes de argumentos e padrões também podem ser usados na mesma definição. Os argumentos nomeados são usados no corpo da função e os padrões de `match` são usados para desestruturar os dados.

Exemplo. Podemos criar uma função que faça uma escolha entre duas opções:

```

def fromOption (default : a) (opt : Option a) : a :=
  match opt with
  | none    => default
  | some x => x

#eval fromOption "vazio" "maca" -- Saída: "maca"
#eval fromOption "vazio" none   -- Saída: "vazio"

-- Definindo sem usar o match
def fromOption (default : a) : Option a -> a
  | none    => default
  | some x => x

#eval fromOption "vazio" (some "maca") -- Saída: "maca"

```

9.3 Definições Locais

De forma geral, é útil nomear etapas intermediárias na criação de um programa. Em muitos casos, a criação de definições intermediárias tornam o programa mais fácil de ler e, além disso, facilitam o uso de valores intermediários que podem aparecer mais de uma vez.

No Lean escrever o mesmo código duas ou mais vezes ou correr o mesmo código duas ou mais vezes faz com que valores sejam computados duas ou mais vezes, retardando o processo.

Exemplo. Vamos definir uma função que transforma uma lista de pares em um par de listas:

```

def unzip : List (a x b) -> List a x List b
  | []          => ([], [])
  | (x, y) :: xys =>
    let unzipped : List a x List b := unzip xys
    (x :: unzipped.fst, y :: unzipped.snd)

```

Uma alternativa que pode deixar o código mais fácil de entender:


```
def unzip : List (a x b) -> List a x List b
| []          => ([], [])
| (x, y) :: xys =>
  let (xs, ys) : List a x List b := unzip xys
  (x :: xs, y :: ys)
```

Observação: Toda vez que aplicamos `unzip` ele chama a definição duas vezes, uma relacionada a `fst` e outra a `snd`. No caso da primeira componente estamos descartando o que acontece em `snd`.

Observe que dessa forma o Lean está fazendo a chamada recursiva duas vezes. Fazer a recursão duas vezes, ou seja, repetir a recursão pode ser evitado. É possível escrever o código de forma tal que o Lean faça a recursão apenas uma vez.

No **Lean** o resultado da chamada recursiva pode ser nomeado e, assim, salvo, usando `let`. Definições locais usando `let` se assemelham a definições de nível superior usando `def`, no seguinte sentido: a estrutura da definição com `let` e com `def` é a mesma. A definição deve de ser nomeada, ela recebe argumentos se desejado, uma assinatura de tipo e, depois de `:=`, o corpo da definição.

Exemplo.

```
def unzip : List (a x b) -> List a x List b
| []          => ([], [])
| (x, y) :: xys =>
  let unzipped : List a x List b := unzip xys
  (x :: unzipped.fst, y :: unzipped.snd)
```

A maior diferença entre `let` e `def` é que definições recursivas com `let` serão explicitamente indicadas por `let rec`.

Exemplo. Vamos criar uma função que inverte a ordem dos elementos de uma lista:

```
def reverse (xs : List a) : List a :=
  let rec helper : List a -> List a -> List a
  | [],      outro => outro
```

```
| y :: ys, outro => helper ys (y :: outro)
helper xs []
```

Vejamos como funciona `helper`:

$$\begin{aligned} \text{helper } [1, 2, 3] [] &\rightarrow \text{helper } [2, 3] [1] \\ &\rightarrow \text{helper } [3] [2, 1] \\ &\rightarrow \text{helper } [] [3, 2, 1] \\ &= [3, 2, 1] \end{aligned}$$

9.4 If Let

No **Lean** existe um tipo indutivo especial chamado `Option`:

```
inductive Option (a : Type) where
| none : Option a
| some : a -> Option a
```

Este tipo representa a ideia de um valor que pode existir ou não.

O `if let` é uma expressão de sintaxe do Lean usada para desestruturar um `Option` e executar um bloco só se fizer match.

Exemplo.

```
def mostra (s : Option String) : String :=
  if let some x := s then
    "Contem" ++ x
  else
    "Vazio"

#eval mostra "ola" -- Saída: "Contem ola"
#eval mostra none  -- Saída: "Vazio"
```

Exemplo. Função que encontra o primeiro número par de uma lista:

```
def PrimeiroPar (xs : List Nat) : Option Nat :=
  xs.find? (fun x => x % 2 == 0)
```

Observação: `%` é o operador modulo. Assim `x % 2` significa "x módulo 2". A função `xs.find?` procura o primeiro elemento numa lista com uma propriedade específica.

```
def descreverPar (xs : List Nat) : Nat :=
  if let some x := PrimeiroPar xs then
    x
  else
    0

#eval PrimeiroPar [1, 3, 4, 7]    -- Saída: some 4
#eval descreverPar [1, 3, 4, 7]  -- Saída: 4
#eval descreverPar [1, 3, 5]     -- Saída: 0
```

`PrimeiroPar` é uma função que retorna um `Option Nat`. Ou seja, ela retorna “algum valor” (`some valor`) ou nada (`none`). `descreverPar` é uma função que retorna o valor propriamente.

9.5 Interpolação de String

A sintaxe `s!` permite colocar dentro de uma frase ou dentro de uma string um valor de alguma outra natureza.

Exemplo.

```
#eval s!"tres vezes cinco e {3 * 5}"
-- Saída: "tres vezes cinco e 15"
```

Referências

- [1] CHRISTIANSEN, David Thrane. *Functional Programming in Lean*. Microsoft Corporation and Lean FRO, LLC, 2023. Disponível em: https://lean-lang.org/functional_programming_in_lean/. Acesso em: 30 set. 2026.
- [2] LEAN FRO. *Lean 4 Documentation*. 2025. Disponível em: <https://lean-lang.org/lean4/doc/>. Acesso em: 30 set. 2026.